

REFERENCIA DE FUNCIONES

Pipeline OS2 de siniestros viales y dashboard

Todas las funciones del proyecto y de las bibliotecas que usa

Rodrigo Sena

Ingeniero Civil en Geografía, Universidad de Santiago de Chile

Magíster en Data Science, Pontificia Universidad Católica de Chile

Gerente General, SEDRA Ingeniería Ltda.

Proyecto: Siniestros viales (proj-002), área de desarrollo de software

Versión del código: script.py y dashboard/app.py del 7 de octubre de 2026

Documento principal: Documentación técnica del pipeline OS2 de siniestros viales, versión 2

8 de octubre de 2026

1. Presentación

Este documento describe una por una las 93 funciones, clases y métodos del proyecto: 86 en script.py y 7 en dashboard/app.py. Complementa al manual técnico, que explica el proceso por etapas, con una consulta por función, y agrega al final las funciones de bibliotecas externas y de la biblioteca estándar de Python que el código usa, con el propósito de cada una en el proyecto.

Las firmas, la ubicación y la relación de llamadas se obtuvieron analizando el código fuente con el módulo ast de Python, de modo que coinciden exactamente con la versión indicada; las descripciones se redactaron a partir de la lectura de cada función. Si el código cambia, basta volver a ejecutar ese análisis para detectar firmas o llamadas que ya no correspondan.

1.1 Cómo leer una ficha

Cada función tiene una ficha con seis campos. **Qué hace** resume su propósito y su lógica. **Firma** reproduce la definición tal como está en el código, con anotaciones de tipo y valores por defecto. **Parámetros** describe cada argumento y **Devuelve**, el valor de retorno y los errores que lanza a propósito. **Llamada desde** indica qué funciones la usan; cuando la llamada está en el cuerpo principal de una etapa se indica como bloque de esa etapa. **Ubicación** da la línea de inicio en el archivo.

Las funciones con nombre compuesto, como a_canonico.tomar o Geocodificador.nombre, son funciones internas de otra o métodos de una clase. Las que empiezan con guion bajo son de uso interno y no están pensadas para llamarse desde fuera de su módulo o clase. Algunas funciones no tienen llamadas directas porque se pasan como argumento a otra (parse_fecha a aplicar_por_unicos, descargar al ejecutor de descargas paralelas) o porque las invoca una biblioteca (los métodos de los lectores de pyosmium); en esos casos la ficha lo indica.

1.2 Organización del código

Tabla 1. Funciones, clases y métodos por sección

Sección	Elementos	Líneas
Funciones generales	10	115 a 661
Etapla 1: descarga	5	674 a 748
Etapla 2: Excel a CSV	1	798 a 798
Etapla 3: columnas canónicas	3	868 a 882
Etapla 4: normalización	13	973 a 1192
Etapla 6: unión, homologación y revisión	3	1319 a 1376
Etapla 7: dirección única	11	1665 a 2107
Etapla 8: índices y fuentes	19	2229 a 2681
Etapla 8: clase Geocodificador	21	2733 a 3149
Dashboard (dashboard/app.py)	7	91 a 339
Total	93	

Fuente: elaboración propia, análisis del código con el módulo ast.

En script.py cada etapa define sus funciones inmediatamente antes del bloque if DESDE <= N <= HASTA: que la ejecuta, por lo que el orden de las fichas sigue el orden del proceso. Las constantes de configuración que usan estas funciones se resumen en el último capítulo.

2. Funciones generales

Funciones de apoyo que usan varias etapas: lectura de argumentos, formato de cifras, huellas, lectura y escritura segura de CSV y descubrimiento de archivos.

2.0.1 `_arg_int`

Qué hace	Lee de la línea de comandos el entero que sigue a una opción. Se evalúa al cargar el script para fijar DESDE, HASTA y SEMILLA_MUESTRA.
Firma	<code>_arg_int(nombre: str, defecto: int) -> int</code>
Parámetros	nombre: opción buscada, por ejemplo <code>--desde</code> ; defecto: valor si la opción no se indicó o no trae número.
Devuelve	El entero indicado o defecto.
Llamada desde	nivel de módulo.
Ubicación	<code>script.py</code> , línea 115

2.0.2 `duracion`

Qué hace	Da formato a un tiempo transcurrido para las bitácoras.
Firma	<code>duracion(seg: float) -> str</code>
Parámetros	seg: segundos.
Devuelve	Texto con la forma 3 min 05 s.
Llamada desde	bloque de la etapa 6; bloque de la etapa 7; bloque de la etapa 8; bloque de la etapa 9; <code>construir_indice</code> ; <code>descargar_mop</code> ; <code>indice_mop</code> .
Ubicación	<code>script.py</code> , línea 606

2.0.3 `miles`

Qué hace	Escribe un número entero con punto como separador de miles, la convención chilena que usan todas las bitácoras.
Firma	<code>miles(n) -> str</code>
Parámetros	n: número; se trunca a entero.
Devuelve	Texto, por ejemplo 1.275.080.
Llamada desde	bloque de la etapa 4; bloque de la etapa 6; bloque de la etapa 7; bloque de la etapa 8; bloque de la etapa 9; <code>construir_indice</code> ; <code>descargar_mop</code> ; <code>homologar_tipo_siniestro</code> ; <code>indice_mop</code> ; <code>normalizar_archivo</code> ; <code>perfil</code> ; <code>resolver_territorio</code> .
Ubicación	<code>script.py</code> , línea 611

2.0.4 `sha256_bytes`

Qué hace	Calcula la huella SHA-256 del contenido recién descargado, que se guarda en el manifiesto para detectar si Carabineros reemplazó una planilla.
Firma	<code>sha256_bytes(data: bytes) -> str</code>
Parámetros	data: contenido binario.
Devuelve	Huella hexadecimal de 64 caracteres.
Llamada desde	<code>descargar</code> .
Ubicación	<code>script.py</code> , línea 615

2.0.5 `sha256_archivo`

Qué hace	Calcula la huella SHA-256 de un archivo ya presente en disco, leyéndolo en bloques de 64 KiB para no cargarlo entero en memoria. Se usa cuando <code>--skip-existing</code> omite la descarga.
Firma	<code>sha256_archivo(path: Path) -> str</code>
Parámetros	path: ruta del archivo.
Devuelve	Huella hexadecimal de 64 caracteres.
Llamada desde	<code>descargar</code> .
Ubicación	<code>script.py</code> , línea 619

2.0.6 nombre_clave

Qué hace	Reduce un encabezado de columna a una clave comparable entre años: minúsculas, saltos de línea y tabulaciones como espacio, sin tildes, todo signo reemplazado por guion bajo y sin guiones bajos repetidos ni en los extremos. Así Tipo Accdte, TIPO_ACCDTE y Tipo-Accdte dan tipo_accdte.
Firma	nombre_clave(name: str) -> str
Parámetros	name: encabezado original.
Devuelve	Clave en minúsculas.
Llamada desde	a_canonico.
Ubicación	script.py, línea 627

2.0.7 escribir_csv_seguro

Qué hace	Escribe una tabla como CSV del pipeline (punto y coma, utf-8-sig, sin índice) en un archivo temporal .tmp y luego lo pone en su lugar con os.replace, que es atómico. Si el destino está bloqueado por OneDrive o Excel reintenta seis veces con cinco segundos de espera, registrando cada intento.
Firma	escribir_csv_seguro(df: pd.DataFrame, destino: Path) -> None
Parámetros	df: tabla a escribir; destino: ruta final.
Devuelve	Nada. Lanza PermissionError con una indicación para el usuario si no logra reemplazar el archivo.
Llamada desde	bloque de la etapa 2; bloque de la etapa 3; bloque de la etapa 4; bloque de la etapa 5; bloque de la etapa 6; bloque de la etapa 7; bloque de la etapa 8; cargar_alias_manual.
Ubicación	script.py, línea 636

2.0.8 leer_csv

Qué hace	Lee un CSV escrito por el pipeline con todas las columnas como texto, de modo que ningún valor cambie de tipo sin que el código lo pida.
Firma	leer_csv(path: Path) -> pd.DataFrame
Parámetros	path: ruta del CSV.
Devuelve	DataFrame con columnas de texto.
Llamada desde	bloque de la etapa 3; bloque de la etapa 4; bloque de la etapa 5; bloque de la etapa 6; bloque de la etapa 7; bloque de la etapa 8; bloque de la etapa 9.
Ubicación	script.py, línea 653

2.0.9 sufijo_perodo

Qué hace	Arma el sufijo de periodo de los nombres de archivo.
Firma	sufijo_perodo(anio: int, mes: str None) -> str
Parámetros	anio: año; mes: mes con dos dígitos, o None para un archivo anual.
Devuelve	2024 o 2026_03.
Llamada desde	bloque de la etapa 2; destino_raw.
Ubicación	script.py, línea 657

2.0.10 listar_archivos

Qué hace	Recorre una carpeta y conserva los archivos cuyo nombre cumple el patrón <grupo>_<año>[_<mes>].<ext> (RE_ARCHIVO) con una extensión permitida. Es la forma en que cada etapa descubre su entrada.
Firma	listar_archivos(carpeta: Path, ext: tuple[str, ...]) -> list[tuple[Path, str, int, str None]]
Parámetros	carpeta: carpeta de entrada de la etapa; ext: extensiones aceptadas, sin punto.
Devuelve	Lista de tuplas (ruta, grupo, año, mes o None) ordenada por grupo (accidentes, personas, vehículos), año y mes.
Llamada desde	bloque de la etapa 2; bloque de la etapa 3; bloque de la etapa 4; bloque de la etapa 5; bloque de la etapa 6; bloque de la etapa 7.
Ubicación	script.py, línea 661

3. Etapa 1: descarga

Funciones que leen el portal de Carabineros, eligen un archivo por grupo y periodo y descargan las planillas.

3.0.1 *fetch_html*

Qué hace	Descarga la página índice del portal OS2 con hasta tres intentos y una espera de 3, 6 y 9 segundos entre ellos.
Firma	<code>fetch_html(url: str) -> str</code>
Parámetros	url: dirección de la página índice (PAGE_URL).
Devuelve	El HTML como texto. Lanza <code>RuntimeError</code> si fallan todos los intentos.
Llamada desde	bloque de la etapa 1.
Ubicación	script.py, línea 674

3.0.2 *parse_candidatos*

Qué hace	Extrae del HTML todos los enlaces a planillas OS2/... (xlsx, xls o xlsb) y clasifica cada uno con tres patrones: mensual (RE_MENSUAL), consolidado por carpeta de año (RE_CONSOL, prioridad 2) e histórico con versión opcional (RE_HIST). Asigna el grupo según el prefijo y descarta los años fuera del rango 2010 al año en curso.
Firma	<code>parse_candidatos(html: str) -> list[dict]</code>
Parámetros	html: contenido de la página índice.
Devuelve	Lista de diccionarios con href, grupo, año, mes, versión y prioridad. Lanza <code>RuntimeError</code> si no hay enlaces, señal de un cambio del portal.
Llamada desde	bloque de la etapa 1.
Ubicación	script.py, línea 686

3.0.3 *seleccionar*

Qué hace	Elige un solo archivo por grupo y periodo. Para cada grupo y año conserva el anual de mayor prioridad y, a igual prioridad, el de mayor versión; luego agrega las planillas mensuales solo de los años que no tienen anual, e informa en la bitácora los meses disponibles.
Firma	<code>seleccionar(cand: list[dict]) -> list[dict]</code>
Parámetros	cand: candidatos de <code>parse_candidatos</code> .
Devuelve	Lista ordenada por año, grupo y mes de los archivos a descargar.
Llamada desde	bloque de la etapa 1.
Ubicación	script.py, línea 716

3.0.4 *destino_raw*

Qué hace	Calcula la ruta local de una planilla, conservando su extensión original.
Firma	<code>destino_raw(item: dict) -> Path</code>
Parámetros	item: candidato seleccionado.
Devuelve	Ruta <code>data/raw/<grupo>_<periodo>.<ext></code> .
Llamada desde	descargar.
Ubicación	script.py, línea 743

3.0.5 *descargar*

Qué hace	Descarga una planilla y la guarda en <code>data/raw/</code> . Rechaza respuestas menores a 1.000 bytes, que suelen ser páginas de error servidas con código 200, y reintenta tres veces. Con <code>--skip-existing</code> no descarga si el archivo ya existe y solo calcula su huella. Se ejecuta en paralelo, seis a la vez, mediante <code>ThreadPoolExecutor</code> .
Firma	<code>descargar(item: dict) -> dict</code>
Parámetros	item: candidato seleccionado.
Devuelve	Fila del manifiesto: grupo, año, mes, enlace, URL, ruta local, estado (descargado, omitido o failed: ...), bytes y huella.
Llamada desde	bloque de la etapa 1 (a través de <code>ThreadPoolExecutor.submit</code>).
Ubicación	script.py, línea 748

4. Etapa 2: Excel a CSV

Lectura de las planillas con el motor que corresponde a cada formato.

4.0.1 leer_excel

Qué hace	Lee la primera hoja de una planilla con todos los valores como texto, eligiendo el motor según la extensión (openpyxl, pyxlsb o xlrd, en ENGINE_MAP).
Firma	leer_excel(path: Path) -> pd.DataFrame
Parámetros	path: ruta de la planilla.
Devuelve	DataFrame con encabezados de la primera fila. Lanza ValueError si la extensión no está soportada.
Llamada desde	bloque de la etapa 2.
Ubicación	script.py, línea 798

5. Etapa 3: columnas canónicas

Traducción de cada planilla a un esquema fijo, sin modificar los valores.

5.0.1 limpiar_vacios

Qué hace	Quita espacios en los extremos y convierte en nulos los valores que solo representan ausencia (nan, none, <na>, null y la cadena vacía, en VACIOS).
Firma	limpiar_vacios(s: pd.Series) -> pd.Series
Parámetros	s: columna de origen.
Devuelve	Columna de texto con nulos reales.
Llamada desde	a_canonico.tomar.
Ubicación	script.py, línea 868

5.0.2 a_canonico

Qué hace	Traduce una planilla al esquema fijo de su grupo (S3_SCHEMA) sin modificar los valores. Agrupa los encabezados por su clave, toma cada campo canónico de la primera columna disponible entre sus sinónimos (ALIAS) y resuelve los casos que dependen del año: el par de columnas de comuna (código y nombre en cinco variantes), el par de causa, y la columna Sector, que desde 2025 trae la condición urbano o rural. Agrega anio_archivo y mes_archivo.
Firma	a_canonico(df: pd.DataFrame, grupo: str, anio: int, mes: str None)
Parámetros	df: planilla convertida a CSV; grupo: accidentes, personas o vehiculos; anio: año del archivo; mes: mes del archivo o None.
Devuelve	Tupla (tabla canónica, lista de mapeos origen -> canónico aplicados, diccionario de columnas desconocidas).
Llamada desde	bloque de la etapa 3.
Ubicación	script.py, línea 873

5.0.3 a_canonico.tomar

Qué hace	Función interna de a_canonico. Busca la primera clave de la lista que exista en el archivo, la marca como usada (para detectar después las columnas desconocidas), registra el mapeo y devuelve la columna limpia.
Firma	tomar(canon: str, *claves) -> pd.Series
Parámetros	canon: nombre del campo canónico; *claves: sinónimos en orden de preferencia.
Devuelve	Columna limpia con limpiar_vacios, o una columna vacía si no existe ninguna.
Llamada desde	a_canonico.
Ubicación	script.py, línea 882

6. Etapa 4: normalización

Limpieza de texto, fecha y hora, y resolución del territorio con los Códigos Únicos Territoriales.

6.0.1 normalizar_texto

Qué hace	Deja un texto en forma canónica. Si detecta los caracteres Ã o Ä, típicos de UTF-8 leído como cp1252 o latin-1, reintentará la decodificación. Luego repara patrones dañados observados en las planillas antiguas (Castaiz 1/2os a Castaños, Henri¿Quez a Henríquez, Ermita±Os, Quell%4n, O·Higgins a O'Higgins), quita los signos sueltos al inicio, elimina tildes por descomposición NFKD (la ñ queda como n), colapsa espacios y aplica tipo título con minúscula tras un dígito (5a y no 5A).
Firma	normalizar_texto(x)
Parámetros	x: valor de cualquier tipo.
Devuelve	Texto normalizado, o None si queda vacío o es un vacío textual.
Llamada desde	bloque de la etapa 4; clave_territorio; normalizar_archivo; resolver_territorio.
Ubicación	script.py, línea 973

6.0.2 aplicar_por_unicos

Qué hace	Aplica una función a cada valor distinto de una columna una sola vez y reconstruye la columna con un diccionario. En columnas con muchos valores repetidos, como comunas o tipos, reduce el trabajo de millones de llamadas a algunos miles.
Firma	aplicar_por_unicos(s: pd.Series, fn) -> pd.Series
Parámetros	s: columna; fn: función de un valor.
Devuelve	Columna de texto transformada.
Llamada desde	normalizar_archivo.
Ubicación	script.py, línea 1007

6.0.3 quitar_sufijo_entero

Qué hace	Elimina el .0 que Excel agrega a los números enteros exportados como texto (12.0 a 12).
Firma	quitar_sufijo_entero(s: pd.Series) -> pd.Series
Parámetros	s: columna.
Devuelve	Columna de texto.
Llamada desde	normalizar_archivo.
Ubicación	script.py, línea 1015

6.0.4 parse_fecha

Qué hace	Interpreta una fecha en tres formas: número de serie de Excel (entre 20.000 y 80.000 días desde el 30 de diciembre de 1899), fecha ISO con guion o barra, y fecha chilena día, mes y año con barra o guion. Una fecha inexistente, como el 31 de febrero, devuelve nulo.
Firma	parse_fecha(x)
Parámetros	x: valor de origen.
Devuelve	Fecha aaaa-mm-dd o None.
Llamada desde	normalizar_archivo (a través de aplicar_por_unicos).
Ubicación	script.py, línea 1019

6.0.5 parse_hora

Qué hace	Interpreta una hora como fracción de día de Excel (0,5 es 12:00, redondeada al minuto) o como texto que contenga hh:mm con valores válidos.
Firma	parse_hora(x)
Parámetros	x: valor de origen.
Devuelve	Hora hh:mm o None.
Llamada desde	normalizar_archivo (a través de aplicar_por_unicos).
Ubicación	script.py, línea 1038

6.0.6 clave_territorio

Qué hace	Normaliza un nombre territorial y lo reduce a letras y dígitos en minúscula, para comparar nombres de comuna sin importar tildes, espacios ni signos.
Firma	clave_territorio(x: str) -> str
Parámetros	x: nombre.
Devuelve	Clave, por ejemplo puntaarenas.
Llamada desde	bloque de la etapa 4; construir_indice; cut_por_nombre; region_desde_texto.
Ubicación	script.py, línea 1053

6.0.7 region_desde_texto

Qué hace	Reconoce la región cuando no se pudo resolver la comuna: acepta un número de uno o dos dígitos, un número romano (ROMANOS) o una palabra clave contenida en el nombre (REGION_CLAVES).
Firma	region_desde_texto(txt)
Parámetros	txt: texto de región de origen.
Devuelve	Código de región de dos dígitos o None.
Llamada desde	resolver_territorio.
Ubicación	script.py, línea 1066

6.0.8 cut_por_codigo

Qué hace	Obtiene el Código Único Territorial a partir de un código numérico de cuatro o cinco dígitos, completando el cero perdido a la izquierda y traduciendo los 21 códigos de Ñuble anteriores a 2018 (OLD_NUBLE).
Firma	cut_por_codigo(v)
Parámetros	v: código de origen.
Devuelve	CUT de cinco dígitos vigente, o None.
Llamada desde	resolver_comuna; resolver_territorio.
Ubicación	script.py, línea 1081

6.0.9 cut_por_nombre

Qué hace	Obtiene el CUT a partir del nombre de la comuna: primero por coincidencia exacta de la clave, luego por alias (ALIAS_COMUNAS) y por último por aproximación con difflib.get_close_matches y umbral 0,85. Un valor puramente numérico no se trata como nombre.
Firma	cut_por_nombre(v)
Parámetros	v: nombre de origen.
Devuelve	Tupla (CUT o None, tipo de coincidencia exacto, alias o aproximado).
Llamada desde	resolver_comuna.
Ubicación	script.py, línea 1093

6.0.10 resolver_comuna

Qué hace	Decide la comuna combinando nombre y código. El nombre manda cuando calza de forma exacta o por alias, porque aparece en todos los años y porque entre 2020 y 2024 el código de origen incluye valores que no son CUT. El código se usa si el nombre falta o solo calza por aproximación, y la aproximación es el último recurso. También prueba los campos cruzados, por si el código vino en la columna de nombre.
Firma	resolver_comuna(cod, nom)
Parámetros	cod: código de origen; nom: nombre de origen.
Devuelve	Tupla (CUT o None, vía usada, si había un código que no es CUT vigente, si código y nombre apuntan a comunas distintas).
Llamada desde	resolver_territorio.
Ubicación	script.py, línea 1107

6.0.11 resolver_territorio

Qué hace	Resuelve el territorio de un archivo completo. Evalúa resolver_comuna una vez por combinación distinta de código, nombre y región, completa región, provincia y comuna con sus códigos desde TABLA_CUT y, si no hay comuna, intenta al menos la región. Registra en la bitácora la cobertura, los códigos inexistentes, las discrepancias, las aproximaciones y los nombres no resueltos más frecuentes.
Firma	resolver_territorio(df: pd.DataFrame, archivo: str) -> pd.DataFrame
Parámetros	df: archivo canónico; archivo: nombre del archivo para la bitácora.
Devuelve	DataFrame con cod_region, region, cod_provincia, provincia, cod_comuna y comuna.
Llamada desde	normalizar_archivo.
Ubicación	script.py, línea 1131

6.0.12 normalizar_archivo

Qué hace	Aplica todas las transformaciones de la etapa 4 a un archivo: identificadores sin sufijo decimal, fecha, hora, mes derivado de la fecha, territorio, columnas de texto del grupo (COLS_TEXTO), zona U y R expandida, número de parte sin signos sueltos, y en personas la unificación de Fallecido en Muerto. Advierte los valores de fecha y hora no interpretables con ejemplos.
Firma	normalizar_archivo(df: pd.DataFrame, grupo: str, archivo: str)
Parámetros	df: archivo canónico; grupo: grupo del archivo; archivo: nombre para la bitácora.
Devuelve	Tupla (tabla con el esquema S4_SCHEMA del grupo, diccionario de celdas modificadas por columna).
Llamada desde	bloque de la etapa 4.
Ubicación	script.py, línea 1188

6.0.13 normalizar_archivo.registrar

Qué hace	Función interna de normalizar_archivo que cuenta cuántas celdas cambió una transformación, comparando antes y después con los nulos como un valor más.
Firma	registrar(col: str, antes: pd.Series, despues: pd.Series)
Parámetros	col: columna; antes: valores de origen; despues: valores transformados.
Devuelve	Nada; acumula el conteo en el diccionario de cambios.
Llamada desde	normalizar_archivo.
Ubicación	script.py, línea 1192

7. Etapa 6: unión, homologación y revisión

Homologación del tipo de siniestro, muestras y perfil de calidad de los consolidados.

7.0.1 homologar_tipo_siniestro

Qué hace	Lleva el tipo de siniestro a seis categorías estándar. Separa el código numérico del texto, toma la primera palabra del texto sin tildes y la traduce con TIPO_PRIMERA_PALABRA; las palabras no reconocidas van a Otros y se advierten. Para las filas que solo traen código, aprende el mapa de código a categoría con una tabla cruzada de las filas que traen ambos, asigna la categoría más frecuente e informa la pureza de cada código.
Firma	homologar_tipo_siniestro(sin: pd.DataFrame) -> pd.DataFrame
Parámetros	sin: consolidado de siniestros.
Devuelve	La misma tabla con tipo_siniestro, cod_tipo_accidente y tipo_accidente resueltos.
Llamada desde	bloque de la etapa 6.
Ubicación	script.py, línea 1319

7.0.2 muestra_por_anio

Qué hace	Toma una muestra aleatoria estratificada por año con la semilla de la corrida, porque cada año tiene un formato de origen distinto, y la imprime en la bitácora traspuesta (una fila por columna y una columna por registro), en bloques y con los valores recortados.
Firma	muestra_por_anio(df: pd.DataFrame, nombre: str, por_anio: int = 1, bloque: int = 6, ancho: int = 26) -> pd.DataFrame
Parámetros	df: tabla; nombre: nombre para el encabezado; por_anio: registros por año; bloque: registros por bloque impreso; ancho: caracteres máximos por valor.
Devuelve	La muestra tomada, como DataFrame.
Llamada desde	bloque de la etapa 6; bloque de la etapa 9.
Ubicación	script.py, línea 1359

7.0.3 perfil

Qué hace	Describe un consolidado columna por columna: porcentaje completo, valores distintos y más frecuentes, y la lista completa de categorías de las columnas con 40 o menos. Emite observaciones de formato: valores no numéricos en columnas numéricas, caracteres no ASCII, codificación rota y espacios sobrantes. Escribe perfil_columnas.csv y observaciones.csv.
Firma	perfil(df: pd.DataFrame, nombre: str)
Parámetros	df: consolidado; nombre: nombre del archivo.
Devuelve	Nada; escribe en la bitácora y en disco.
Llamada desde	bloque de la etapa 6.
Ubicación	script.py, línea 1376

8. Etapa 7: dirección única

Limpieza de nombres de vía, rutas, kilómetros y números, y armado de la dirección.

8.0.1 _tok_titulo

Qué hace	Escribe una palabra en tipo título según su papel: números romanos en mayúscula, partículas (PARTICULAS) en minúscula cuando no van al inicio, artículos en minúscula solo tras una partícula (Paseo de la Paz) y mayúscula tras apóstrofo o guion (O'Higgins, Colo-Colo).
Firma	_tok_titulo(t: str, primero: bool, previo: str = "") -> str
Parámetros	t: palabra; primero: si es la primera palabra; previo: palabra anterior.
Devuelve	Palabra con mayúsculas aplicadas.
Llamada desde	limpiar_vía.
Ubicación	script.py, línea 1665

8.0.2 es_fecha_excel

Qué hace	Indica si un número entero es en realidad una fecha de Excel entre 2000 y 2029 (36.526 a 47.483) escrita en lugar de una altura.
Firma	es_fecha_excel(n) -> bool
Parámetros	n: valor numérico o texto.
Devuelve	True o False.
Llamada desde	armar_direccion.
Ubicación	script.py, línea 1680

8.0.3 limpiar_vía

Qué hace	Limpia un nombre de vía con reglas en orden: descarta vacíos (VACIOS_DIR) y valores que son fechas o números, lleva al frente el tipo escrito como sufijo, quita prefijos de lugar, paréntesis e indicación de sector, extrae un kilómetro escrito dentro del nombre, expande tipos de vía (TIPOS_VIA) y abreviaturas (ABREVIATURAS) y aplica tipo título. Está memorizada con functools.lru_cache, porque los mismos nombres se repiten cientos de miles de veces.
Firma	limpiar_vía(nombre)
Decorador	@functools.lru_cache(maxsize=None)
Parámetros	nombre: nombre de vía de origen.
Devuelve	Tupla (nombre limpio o None, lista de reglas aplicadas, kilómetro encontrado en el texto o None).
Llamada desde	_partir_interseccion; armar_direccion.
Ubicación	script.py, línea 1685

8.0.4 limpiar_ruta

Qué hace	Lleva un rol de ruta a su forma oficial (5, 68, E-71, CH-60). Descarta las rutas sin rol, toma la primera de una ruta múltiple, quita prefijos como Ruta, Rta o Carretera y traduce Longitudinal o Panamericana a 5. Si el texto no es un código, como Acceso Sur, lo devuelve aparte para usarlo como nombre de vía.
Firma	limpiar_ruta(r)
Decorador	@functools.lru_cache(maxsize=None)
Parámetros	r: valor de la columna de ruta.
Devuelve	Tupla (código o None, reglas aplicadas, texto de vía o None).
Llamada desde	armar_direccion; construir_indice; indice_mop; ruta_en_calle.
Ubicación	script.py, línea 1785

8.0.5 ruta_en_calle

Qué hace	Detecta que una calle es en realidad una ruta (Carretera Ruta 68, Autopista 5 Sur, E-71) con un patrón estricto que no confunde las calles numéricas, como 2 Norte en Talca.
Firma	ruta_en_calle(c1)
Parámetros	c1: nombre de calle ya limpio.
Devuelve	Código de ruta o None.
Llamada desde	armar_direccion.
Ubicación	script.py, línea 1814

8.0.6 limpiar_km

Qué hace	Interpreta un kilómetro con coma o punto decimal y sin la palabra km. Acepta valores entre 0 y 3.500.
Firma	limpiar_km(k)
Parámetros	k: valor de la columna de kilómetro.
Devuelve	Kilómetro como texto entero o con un decimal, o None.
Llamada desde	armar_direccion.
Ubicación	script.py, línea 1828

8.0.7 limpiar_numero

Qué hace	Extrae una altura de hasta cinco dígitos con una letra opcional (1234B), ignorando el texto previo; el cero no es una altura.
Firma	limpiar_numero(n)
Parámetros	n: valor de número de frente.
Devuelve	Altura como texto o None.
Llamada desde	armar_direccion.
Ubicación	script.py, línea 1841

8.0.8 sin_tipo

Qué hace	Quita el tipo de vía inicial para comparar nombres equivalentes, como Avenida 5 de Abril y 5 de Abril.
Firma	sin_tipo(v: str) -> str
Parámetros	v: nombre de vía.
Devuelve	Nombre en minúsculas sin el tipo.
Llamada desde	_partir_interseccion; armar_direccion; limpiar_via.
Ubicación	script.py, línea 1850

8.0.9 _partir_interseccion

Qué hace	Separa una intersección escrita dentro de la primera calle (Caupolicán Esquina Ramon Zamora, Las Heras Con Pedro Lagos), solo si ambos lados son vías válidas de al menos tres letras sin el tipo.
Firma	_partir_interseccion(c1)
Parámetros	c1: valor de origen de la primera calle.
Devuelve	Tupla (vía A, vía B, regla aplicada) o None.
Llamada desde	armar_direccion.
Ubicación	script.py, línea 1856

8.0.10 armar_direccion

Qué hace	Construye la dirección única de un siniestro. Resuelve primero las contradicciones entre campos (calle 2 que es un número, número dentro de la calle, lugar en lugar de calle, intersección dentro de una sola columna, calles repetidas, ruta escrita como calle, kilómetro que en realidad son metros o una altura, número usado como kilómetro en rutas, mezcla de kilómetros y alturas desde 2025) y luego elige el formato por prioridad: ruta y kilómetro, intersección, calle y número, vía y kilómetro, ruta sola y calle sola. Se ejecuta una vez por combinación distinta de componentes.
Firma	armar_direccion(c1_raw, c2_raw, r_raw, k_raw, f_raw, ur_raw = None, kamb_raw = None)
Parámetros	c1_raw: calle 1; c2_raw: calle 2; r_raw: ruta; k_raw: kilómetro; f_raw: número de frente; ur_raw: zona urbana o rural; kamb_raw: marca de archivo con kilómetro ambiguo (2025 en adelante).
Devuelve	Tupla (dirección o None, tipo de dirección, reglas aplicadas); los componentes limpios quedan en las columnas via_1, via_2, numero, ruta_cod y km_val.
Llamada desde	bloque de la etapa 7.
Ubicación	script.py, línea 1871

8.0.11 _linea

Qué hace	Función interna del diagnóstico de muestras de la etapa 7: arma una línea legible con año, comuna, zona, componentes de origen, dirección resultante y reglas aplicadas.
Firma	_linea(f) -> str
Parámetros	f: fila de la tabla de componentes.
Devuelve	Texto de una línea.
Llamada desde	bloque de la etapa 7.
Ubicación	script.py, línea 2107

9. Etapa 8: Índices y fuentes

Preparación del índice de OpenStreetMap, de la red vial del MOP y de los alias manuales.

9.0.1 alias_osm

Qué hace	Busca la equivalencia conocida de un nombre (ALIAS_OSM). Los alias que traen regiones solo se aplican dentro de ellas, como Alameda a Libertador Bernardo O'Higgins en las regiones 13 y 6.
Firma	alias_osm(k: str, cut: str)
Parámetros	k: clave del nombre; cut: comuna del siniestro.
Devuelve	Clave de destino o None.
Llamada desde	Geocodificador.candidatos; Geocodificador.nombre.
Ubicación	script.py, línea 2229

9.0.2 clave_osm

Qué hace	Crea la clave de comparación de nombres de calle: reemplaza fracciones (16½ a 16 1/2), quita tildes y signos, pasa a minúsculas, convierte números escritos en palabras a cifras (Dos Norte y 2 Norte dan la misma clave), une la O de los apellidos irlandeses (O'Higgins a ohiggins) y quita hasta dos tipos de vía iniciales.
Firma	clave_osm(nombre) -> str
Parámetros	nombre: nombre de calle.
Devuelve	Clave en minúsculas, vacía si el nombre falta.
Llamada desde	Geocodificador._registrar; Geocodificador.calle; Geocodificador.candidatos; Geocodificador.nombre; Geocodificador.punto_manual; cargar_alias_manual; construir_indice.
Ubicación	script.py, línea 2265

9.0.3 clave_nucleo

Qué hace	Quita títulos y partículas de una clave (PALABRAS_VACIAS), de modo que Presidente Salvador Allende y Salvador Allende coincidan.
Firma	clave_nucleo(k: str) -> str
Parámetros	k: clave.
Devuelve	Clave reducida, o la original si quedaría vacía.
Llamada desde	Geocodificador._contenido; Geocodificador._mapas; Geocodificador._unico; Geocodificador.candidatos; Geocodificador.nombre.
Ubicación	script.py, línea 2288

9.0.4 metros

Qué hace	Distancia aproximada entre dos puntos con proyección equirectangular: corrige la longitud por el coseno de la latitud media y multiplica por 111.320 m por grado. Basta a escala urbana.
Firma	metros(lon1, lat1, lon2, lat2) -> float
Parámetros	lon1, lat1: primer punto; lon2, lat2: segundo punto.
Devuelve	Distancia en metros.
Llamada desde	Geocodificador.cruce; Geocodificador.numero; Geocodificador.ruta_km.
Ubicación	script.py, línea 2294

9.0.5 descargar_osm

Qué hace	Descarga el extracto de Chile de Geofabrik por bloques de 1 MiB a un temporal que luego reemplaza al definitivo. No hace nada si el extracto tiene 30 días o menos (OSM_MAX_DIAS), salvo que se pida --osm-actualizar.
Firma	descargar_osm() -> None
Parámetros	No recibe parámetros.
Devuelve	Nada; deja data/osm/chile-latest.osm.pbf.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 2301

9.0.6 ruta_osmium

Qué hace	Entrega una ruta que libosmium pueda abrir en Windows, que no acepta caracteres no ASCII como la Á de Área en la carpeta de OneDrive. Devuelve la misma ruta si ya es ASCII; si no, la ruta corta 8.3 de Windows; si no existe, un enlace duro o una copia en una carpeta temporal con ruta ASCII.
Firma	ruta_osmium(p: Path) -> str
Parámetros	p: ruta del extracto.
Devuelve	Ruta utilizable como texto. Lanza RuntimeError si no hay ninguna carpeta ASCII disponible.
Llamada desde	construir_indice.
Ubicación	script.py, línea 2318

9.0.7 construir_indice

Qué hace	Construye el índice local de OpenStreetMap. Lee el extracto en dos pasadas con pyosmium (calles, números, hitos y localidades; luego límites comunales), asocia cada polígono comunal al CUT, calcula las comunas vecinas, asigna cada vía a las comunas de su primer, central y último nodo, indexa cada vía por nombre y por Ruta X, detecta los cruces como nodos compartidos por vías de nombre distinto y agrupa números, hitos y localidades por comuna o ruta.
Firma	construir_indice() -> dict
Parámetros	No recibe parámetros.
Devuelve	Diccionario con calles, nombre_osm, nombre_osm_c, rutas, cruces, numeros, hitos, lugares, vecinas, poligonos, fuente y version, que el bloque de la etapa guarda en indice_osm.pkl.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 2365

9.0.8 construir_indice.Lector

Qué hace	Manejador de la primera pasada de pyosmium. Su método node recoge números de casa (addr:housenumber con addr:street), hitos kilométricos (highway=milestone) y localidades (place); su método way recoge las vías de los tipos de VIALES.OSM con nombre o código, los números de casa dibujados como polígono y asocia cada hito a la ruta que lo contiene.
Firma	class Lector(osmium.SimpleHandler)
Parámetros	Ver el constructor.
Devuelve	Listas vías, direcciones, lugares y el diccionario hitos.
Llamada desde	construir_indice (pyosmium invoca sus métodos).
Ubicación	script.py, línea 2379

9.0.9 construir_indice.Lector.__init__

Qué hace	Inicializa las colecciones del lector.
Firma	__init__(self)
Parámetros	No recibe parámetros.
Devuelve	Nada.
Llamada desde	construir_indice.
Ubicación	script.py, línea 2380

9.0.10 construir_indice.Lector.node

Qué hace	Procesa un nodo del extracto, como se describe en la clase.
Firma	node(self, n)
Parámetros	n: nodo de pyosmium.
Devuelve	Nada.
Llamada desde	pyosmium, durante apply_file.
Ubicación	script.py, línea 2384

9.0.11 construir_indice.Lector.way

Qué hace	Procesa una vía del extracto, como se describe en la clase; ignora las vías con nodos sin ubicación.
Firma	way(self, w)
Parámetros	w: vía de pyosmium.
Devuelve	Nada.
Llamada desde	pyosmium, durante apply_file.
Ubicación	script.py, línea 2399

9.0.12 construir_indice.LectorComunas

Qué hace	Manejador de la segunda pasada, separada porque armar polígonos es costoso. Su método area convierte en multipolígono cada límite administrativo de nivel 8 (comunas) y guarda su identificador dpachile:id y su nombre.
Firma	class LectorComunas(osmium.SimpleHandler)
Parámetros	Ver el constructor.
Devuelve	Lista comunas con (identificador, nombre, geometría).
Llamada desde	construir_indice (pyosmium invoca sus métodos).
Ubicación	script.py, línea 2418

9.0.13 construir_indice.LectorComunas.__init__

Qué hace	Inicializa la lista de comunas.
Firma	__init__(self)
Parámetros	No recibe parámetros.
Devuelve	Nada.
Llamada desde	construir_indice.
Ubicación	script.py, línea 2420

9.0.14 construir_indice.LectorComunas.area

Qué hace	Procesa un área del extracto, como se describe en la clase.
Firma	area(self, a)
Parámetros	a: área de pyosmium.
Devuelve	Nada.
Llamada desde	pyosmium, durante apply_file.
Ubicación	script.py, línea 2424

9.0.15 construir_indice.comuna_de

Qué hace	Función interna: asigna a cada punto la comuna cuyo polígono lo contiene, con una consulta vectorizada sobre un árbol espacial; a los que caen justo fuera les asigna la comuna más cercana a menos de 0,01 grados.
Firma	comuna_de(lons, lats)
Parámetros	lons: longitudes; lats: latitudes.
Devuelve	Lista de CUT o None, en el mismo orden.
Llamada desde	construir_indice.
Ubicación	script.py, línea 2469

9.0.16 descargar_mop

Qué hace	Descarga la red vial del MOP desde su servicio ArcGIS por páginas de 1.000 tramos, pidiendo la medida M de cada vértice, con reintentos por página. La guarda en data/mop/ y la reutiliza durante 180 días.
Firma	descargar_mop() -> list
Parámetros	No recibe parámetros.
Devuelve	Lista de tuplas (rol, nombre, lista de caminos con vértices (lon, lat, m)). Lanza RuntimeError si una página falla en todos los intentos.
Llamada desde	indice_mop.
Ubicación	script.py, línea 2585

9.0.17 indice_mop

Qué hace	Organiza los tramos del MOP por comuna y ruta: normaliza el rol con limpiar_ruta, descarta los que no tienen rol utilizable, marca si la medida crece en el sentido de la geometría y asigna cada camino a las comunas que contienen una muestra de sus vértices. Se guarda en disco con una clave que cambia si cambian la red, el extracto OSM o la versión del índice.
Firma	indice_mop(idx_osm: dict) -> dict
Parámetros	idx_osm: índice OSM, del que toma los polígonos comunales.
Devuelve	Diccionario con rutas (comuna, ruta, lista de (medidas, coordenadas, creciente)) y clave.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 2628

9.0.18 leer_csv_usuario

Qué hace	Lee un CSV que el usuario editó a mano, probando codificación UTF-8 y ANSI, y separador punto y coma o coma, hasta encontrar la columna nombre_dato.
Firma	leer_csv_usuario(path: Path) -> pd.DataFrame
Parámetros	path: ruta del archivo.
Devuelve	DataFrame de texto. Lanza ValueError si ningún formato funciona.
Llamada desde	cargar_alias_manual.
Ubicación	script.py, línea 2668

9.0.19 cargar_alias_manual

Qué hace	Lee alias_calles.csv y le incorpora las filas que el usuario completó en alias_sugeridos.csv, reescribiéndolo solo si hubo filas nuevas. Convierte cada fila en un alias de nombre (o un bloqueo, si el destino es un guion, NINGUNO o BLOQUEAR) y, si trae coordenadas dentro de Chile, en un punto manual con su precisión.
Firma	cargar_alias_manual() -> dict
Parámetros	No recibe parámetros.
Devuelve	Diccionario {"alias": {(cut, clave): clave_osm o bloqueo}, "puntos": {(cut, clave): (lon, lat, precisión)}}; un CUT vacío vale para todas las comunas.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 2681

10. Etapa 8: clase Geocodificador

Coincidencia de nombres y métodos de ubicación. Los métodos cuyo nombre empieza con guion bajo son de uso interno de la clase.

10.0.1 Geocodificador

Qué hace	Clase que resuelve las direcciones de la etapa 7 contra los índices, siempre dentro de la comuna del siniestro o sus vecinas. Concentra la cadena de coincidencia de nombres, los métodos de ubicación y los contadores de diagnóstico que alimentan la bitácora y alias_sugeridos.csv.
Firma	class Geocodificador
Parámetros	Ver el constructor.
Devuelve	Una instancia por corrida de la etapa 8.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 2733

10.0.2 Geocodificador.__init__

Qué hace	Guarda los índices OSM y MOP y los alias manuales, construye el árbol espacial de polígonos comunales, la lista de claves por comuna, los cachés (nombres, mapas por comuna, contenidos, variantes, geometrías) y los contadores vias_tipo, aproximados, sin_pareja y alias_invalidos.
Firma	__init__(self, idx: dict, mop: dict None = None, manual: dict None = None)
Parámetros	idx: índice OSM; mop: índice MOP o None; manual: alias y puntos manuales o None.
Devuelve	Nada.
Llamada desde	bloque de la etapa 8 (al crear la instancia).
Ubicación	script.py, línea 2736

10.0.3 Geocodificador._comunas

Qué hace	Lista de búsqueda de una comuna: ella misma y luego sus vecinas en orden.
Firma	_comunas(self, cut)
Parámetros	cut: comuna del siniestro.
Devuelve	Lista de CUT.
Llamada desde	Geocodificador.calle; Geocodificador.cruce; Geocodificador.km_mop; Geocodificador.nombre; Geocodificador.ruta_km; bloque de la etapa 8.
Ubicación	script.py, línea 2759

10.0.4 Geocodificador._mapas

Qué hace	Construye y memoriza, la primera vez que se consulta una comuna, tres mapas: nombres sin espacios a claves, núcleos a claves y un índice inverso de palabras de cuatro letras o más a núcleos.
Firma	_mapas(self, c)
Parámetros	c: comuna.
Devuelve	Tupla (compacto, núcleo, inverso).
Llamada desde	Geocodificador._contenido; Geocodificador._iniciales; Geocodificador.candidatos; Geocodificador.nombre.
Ubicación	script.py, línea 2762

10.0.5 Geocodificador._contenido

Qué hace	Busca una calle cuyo núcleo contiene al del dato o al revés (Prat y Arturo Prat). Exige compartir una palabra distintiva de cuatro letras que no sea genérica, los mismos números y direccionales, conservar el apellido cuando el nombre OSM es más corto y no agregar ni quitar títulos que cambien la persona (Vicuña Mackenna no es General Mackenna). Prefiere los candidatos con menos palabras de diferencia.
Firma	_contenido(self, k, c, todos = False)
Parámetros	k: clave del dato; c: comuna; todos: si se quieren todos los candidatos para probar cruces.
Devuelve	Una clave o None; con todos, lista de hasta seis claves.
Llamada desde	Geocodificador.candidatos; Geocodificador.nombre.
Ubicación	script.py, línea 2776

10.0.6 Geocodificador._unico

Qué hace	Reduce una lista de candidatos a uno: el único, o el de más tramos si todos comparten el mismo núcleo y son, por lo tanto, la misma calle.
Firma	_unico(self, claves, c)
Parámetros	claves: candidatos; c: comuna.
Devuelve	Clave o None si hay ambigüedad.
Llamada desde	Geocodificador._contenido; Geocodificador._iniciales; Geocodificador.nombre.
Ubicación	script.py, línea 2830

10.0.7 Geocodificador._iniciales

Qué hace	Resuelve nombres escritos con iniciales (V Mackenna, P A Cerda, J J Perez). Todas las palabras largas deben estar en el candidato, los números deben coincidir y cada inicial debe corresponder a una palabra distinta del nombre OSM.
Firma	_iniciales(self, k, c)
Parámetros	k: clave del dato; c: comuna.
Devuelve	Clave única o None.
Llamada desde	Geocodificador.nombre.
Ubicación	script.py, línea 2840

10.0.8 Geocodificador.candidatos

Qué hace	Reúne hasta ocho claves plausibles para una calle en una comuna (exacta, alias, sin espacios, núcleo y contenido), que se usan para buscar cruces con nombres alternativos.
Firma	candidatos(self, via, c)
Parámetros	via: nombre de la calle; c: comuna.
Devuelve	Lista de claves sin repetir.
Llamada desde	Geocodificador.cruce.
Ubicación	script.py, línea 2874

10.0.9 Geocodificador._aproximado

Qué hace	Coincidencia aproximada con rapidfuzz.process.extract y la medida token_sort_ratio, con umbral 88 (95 si la clave tiene siete caracteres o menos). Entre los tres mejores, acepta el primero con los mismos números, direccionales y letras sueltas.
Firma	_aproximado(self, k, c)
Parámetros	k: clave del dato; c: comuna.
Devuelve	Clave o None.
Llamada desde	Geocodificador.nombre.
Ubicación	script.py, línea 2884

10.0.10 Geocodificador.nombre

Qué hace	Traduce un nombre del parte a una calle del índice. Prueba primero el alias manual, que puede bloquear; luego, en la comuna y en sus vecinas, los pasos exacto, alias regional, sin espacios, núcleo, contenido, iniciales y sin direccional; por último, solo en la comuna del siniestro, la coincidencia aproximada. Memoriza el resultado y cuenta cómo se encontró cada nombre.
Firma	nombre(self, via, cut, n = 1)
Parámetros	via: nombre de la calle; cut: comuna del siniestro; n: filas que representa la consulta, para los contadores.
Devuelve	Tupla (clave OSM o None, comuna donde se encontró, paso que acertó).
Llamada desde	Geocodificador.calle; Geocodificador.cruce; Geocodificador.numero.
Ubicación	script.py, línea 2901

10.0.11 Geocodificador.punto_manual

Qué hace	Busca una coordenada cargada a mano para la calle en la comuna del siniestro o, si no hay, la cargada sin comuna, que vale para todas.
Firma	punto_manual(self, via, cut)
Parámetros	via: nombre; cut: comuna.
Devuelve	Tupla (lon, lat, precisión) o None.
Llamada desde	Geocodificador._registrar; Geocodificador.calle; Geocodificador.numero.
Ubicación	script.py, línea 2973

10.0.12 Geocodificador._registrar

Qué hace	Acumula para el diagnóstico las calles sin pareja (si no hay bloqueo ni punto manual) y las aceptadas por pasos no exactos, con su número de filas y un ejemplo de cómo venían escritas.
Firma	_registrar(self, via, cut, res, n)
Parámetros	via: nombre; cut: comuna; res: resultado de nombre; n: filas.
Devuelve	Nada.
Llamada desde	Geocodificador.calle; Geocodificador.cruce; Geocodificador.numero.
Ubicación	script.py, línea 2977

10.0.13 Geocodificador._punto_medio_calle

Qué hace	Calcula el punto que está a la mitad de la longitud total de todas las geometrías de una calle en la comuna.
Firma	_punto_medio_calle(self, k, c)
Parámetros	k: clave; c: comuna.
Devuelve	Tupla (lon, lat) o None.
Llamada desde	Geocodificador.calle; Geocodificador.numero.
Ubicación	script.py, línea 2986

10.0.14 Geocodificador.variantes

Qué hace	Busca nombres OSM de la comuna que contienen la calle como palabras completas y con a lo sumo dos palabras más (Quilin Sur, Rotonda Quilin), usando un índice inverso de palabras memorizado.
Firma	variantes(self, k, c)
Parámetros	k: clave; c: comuna.
Devuelve	Lista con la clave original y hasta ocho variantes.
Llamada desde	Geocodificador.cruce.
Ubicación	script.py, línea 2994

10.0.15 Geocodificador._geom

Qué hace	Une las geometrías de varias claves en un MultiLineString y lo memoriza, vaciando el caché si supera 200.000 entradas.
Firma	_geom(self, c, claves)
Parámetros	c: comuna; claves: claves a unir.
Devuelve	Geometría o None.
Llamada desde	Geocodificador.cruce.
Ubicación	script.py, línea 3012

10.0.16 Geocodificador.cruce

Qué hace	Ubica una intersección. Resuelve ambos nombres y busca, en la comuna y sus vecinas, un cruce exacto; luego un cruce entre variantes; luego el punto medio entre los puntos más cercanos de ambas calles si están a 40 m o menos (pasos a desnivel); y por último un cruce entre nombres alternativos plausibles.
Firma	cruce(self, v1, v2, cut, n)
Parámetros	v1, v2: calles; cut: comuna; n: filas.
Devuelve	Tupla (lon, lat, método, resultado de la calle 1, resultado de la calle 2) o None.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 3023

10.0.17 Geocodificador.numero

Qué hace	Ubica una calle con número: número exacto; interpolación entre los vecinos de la misma vereda a 200 números o menos y menos de 1 km; interpolación en cualquier vereda hasta 1.000 números y 2,5 km; número cercano a 50 o menos; y, si nada funciona, el punto medio de la calle. Si la calle no existe, usa un punto manual.
Firma	numero(self, via, num, cut, n)
Parámetros	via: calle; num: altura; cut: comuna; n: filas.
Devuelve	Tupla (lon, lat, método, resultado de la calle) o None.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 3062

10.0.18 Geocodificador.calle

Qué hace	Ubica una calle sin número o un sector: si empieza con Sector busca la localidad; si no, el punto medio de la calle o un punto manual.
Firma	calle(self, via, cut, n)
Parámetros	via: calle o sector; cut: comuna; n: filas.
Devuelve	Tupla (lon, lat, método, resultado) o None.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 3095

10.0.19 Geocodificador._comuna_punto

Qué hace	Comuna que contiene un punto, o la más cercana a menos de 0,01 grados.
Firma	_comuna_punto(self, lon, lat)
Parámetros	lon, lat: punto.
Devuelve	CUT o None.
Llamada desde	Geocodificador.km_mop.
Ubicación	script.py, línea 3110

10.0.20 Geocodificador.km_mop

Qué hace	Interpola sobre la red MOP el punto en que la medida M vale el kilómetro por 1.000. En tramos de medida creciente usa búsqueda binaria; en los demás revisa cada segmento. Acepta el primer candidato que cae en la comuna o sus vecinas, porque el mismo kilómetro existe en otras partes de la ruta.
Firma	km_mop(self, ruta, kmf, cut)
Parámetros	ruta: código de ruta; kmf: kilómetro como número; cut: comuna.
Devuelve	Tupla (lon, lat) o None.
Llamada desde	Geocodificador.ruta_km.
Ubicación	script.py, línea 3120

10.0.21 Geocodificador.ruta_km

Qué hace	Ubica una ruta con o sin kilómetro: hito exacto de OSM (a menos de 50 m del kilómetro), red MOP ajustada a la geometría OSM de la ruta si queda a menos de 1.500 m, interpolación entre hitos separados por 20 km o menos, y punto medio de la ruta en la comuna.
Firma	ruta_km(self, ruta, km, cut)
Parámetros	ruta: código; km: kilómetro como texto o None; cut: comuna.
Devuelve	Tupla (lon, lat, método) o None.
Llamada desde	bloque de la etapa 8.
Ubicación	script.py, línea 3149

11. Dashboard

dashboard/app.py tiene pocas funciones porque Streamlit ejecuta el archivo completo de arriba abajo en cada interacción; el resto del código son bloques, uno por pestaña, que leen la tabla filtrada. Las fichas siguientes cubren todas sus funciones.

11.0.1 fmt

Qué hace	Escribe un número con punto de miles y coma decimal, como se usa en Chile.
Firma	fmt(n, dec: int = 0) -> str
Parámetros	n: número; dec: decimales.
Devuelve	Texto, o un guión si el valor es nulo.
Llamada desde	barras; nivel de módulo.
Ubicación	dashboard/app.py, línea 91

11.0.2 rgb

Qué hace	Convierte un color hexadecimal en la lista de cuatro enteros que usa pydeck.
Firma	rgb(hexa: str, alfa: int = 255) -> list[int]
Parámetros	hexa: color #rrggbb; alfa: opacidad de 0 a 255.
Devuelve	Lista [r, g, b, a].
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 98

11.0.3 cargar

Qué hace	Lee y une las partes Parquet, vuelve a categorizar las columnas de texto repetitivo, agrega ksi, lesionados, siniestros y mes_inicio, y lee metadatos.json. Está decorada con st.cache_resource, de modo que se ejecuta una vez por proceso y todas las sesiones comparten la misma tabla sin copiarla.
Firma	cargar() -> tuple[pd.DataFrame, dict]
Decorador	@st.cache_resource(show_spinner='Cargando siniestros...')
Parámetros	No recibe parámetros.
Devuelve	Tupla (tabla de siniestros, diccionario de metadatos).
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 109

11.0.4 leyenda_gravedad

Qué hace	Dibuja con HTML la leyenda de colores del mapa de puntos: con fallecidos, con lesionados graves y sin fallecidos ni graves.
Firma	leyenda_gravedad() -> None
Parámetros	No recibe parámetros.
Devuelve	Nada.
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 216

11.0.5 vista

Qué hace	Calcula la vista inicial de un mapa: centro en la mediana de las coordenadas y zoom a partir de la extensión entre los percentiles 5 y 95, limitado entre 3,6 y 15.
Firma	vista(lat: pd.Series, lon: pd.Series) -> pdk.ViewState
Parámetros	lat: latitudes; lon: longitudes.
Devuelve	pydeck.ViewState.
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 224

11.0.6 eje_y

Qué hace	Crea el eje vertical de los gráficos de Altair con separador de miles chileno y cinco marcas.
Firma	eje_y(titulo: str = "") -> alt.Y
Parámetros	titulo: título del eje, opcional.
Devuelve	alt.Y.
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 282

11.0.7 barras

Qué hace	Crea un gráfico de barras horizontales ordenado de mayor a menor, con el valor escrito al final de cada barra en color de texto y el porcentaje en la ficha emergente.
Firma	barras(datos: pd.DataFrame, campo: str, titulo: str) -> alt.Chart
Parámetros	datos: tabla con la columna valor; campo: columna de categorías; titulo: título.
Devuelve	Gráfico de Altair (barras más texto).
Llamada desde	nivel de módulo.
Ubicación	dashboard/app.py, línea 339

12. Funciones de bibliotecas externas y estándar

Este capítulo lista las funciones de otras bibliotecas que el proyecto usa, agrupadas por biblioteca, con su propósito en el proyecto y la etapa en que aparecen. El inventario se obtuvo del análisis del código, completado con los métodos de tablas y geometrías que se invocan sobre objetos y que ese análisis no distingue por sí solo.

12.1 pandas

Manejo de tablas en memoria; es la base de todas las etapas y del dashboard.

Tabla 2. Funciones de pandas usadas en el proyecto

Función	Uso en el proyecto	Dónde
pd.read_excel	Lee la primera hoja de cada planilla como texto.	etapa 2
pd.read_csv	Lee los CSV del pipeline y los archivos editados a mano.	etapas 3 a 9
pd.read_parquet	Lee cada parte de datos del dashboard.	dashboard
DataFrame.to_csv	Escribe los CSV intermedios, consolidados y de diagnóstico.	todas las etapas
DataFrame.to_parquet	Escribe las partes del dashboard con compresión zstd nivel 19.	etapa 9
pd.concat	Une archivos de un grupo, partes de datos y tablas de resumen.	etapas 5 a 9, dashboard
DataFrame.merge	Une los resultados calculados por combinación única con las filas originales.	etapas 6 a 9
DataFrame.groupby	Agrega por periodo, llave, comuna, dirección y método.	etapas 5 a 9, dashboard
pd.crosstab	Cruza código y categoría de tipo de siniestro para aprender su mapa.	etapa 6
Series.value_counts	Cuenta frecuencias para las bitácoras y para las consultas distintas.	etapas 6 a 9
DataFrame.sample	Toma las muestras aleatorias con la semilla de la corrida.	etapas 6 a 9
Series.map, Series.where, Series.mask, Series.combine_first, Series.fillna	Reemplazos vectorizados y valores por defecto.	todas las etapas
Series.str.* (replace, fullmatch, normalize, lower, split)	Operaciones de texto vectorizadas.	etapas 3 a 7
pd.to_numeric, pd.to_datetime	Conversiones de tipo con valores no interpretables como nulos.	etapas 5 y 9
Series.astype("category")	Columnas de texto repetitivo como categorías, para ahorrar memoria.	etapa 9, dashboard
pd.isna, pd.notna	Pruebas de nulo que aceptan cualquier tipo.	etapas 4 a 8, dashboard

12.2 numpy

Cálculo numérico vectorizado en el dashboard.

Tabla 3. Funciones de numpy usadas en el proyecto

Función	Uso en el proyecto	Dónde
np.select	Asigna la gravedad de cada punto del mapa (fallecidos, graves, resto).	dashboard
np.where	Marca el año parcial en la evolución anual.	dashboard
np.clip, np.log2	Calculan el zoom inicial del mapa.	dashboard
np.sqrt	Radio de los círculos proporcionales de puntos críticos.	dashboard
np.int8, np.isnan	Columna constante compacta y prueba de nulo en fmt.	dashboard

12.3 requests

Descargas por HTTP.

Tabla 4. Funciones de requests usadas en el proyecto

Función	Uso en el proyecto	Dónde
requests.get	Descarga la página índice, las planillas, el extracto OSM (en modo stream) y las páginas de la red MOP.	etapas 1 y 8

12.4 pyosmium (osmium)

Lectura del extracto de OpenStreetMap en formato PBF.

Tabla 5. Funciones de pyosmium (osmium) usadas en el proyecto

Función	Uso en el proyecto	Dónde
osmium.SimpleHandler	Clase base de los lectores Lector y LectorComunas.	etapa 8
SimpleHandler.apply_file	Recorre el extracto con ubicaciones de nodos (locations=True, índice flex_mem).	etapa 8
osmium.filter.KeyFilter	Descarta en C++ los objetos sin las etiquetas útiles, lo que acelera la primera pasada.	etapa 8
osmium.io.File	Abre el extracto indicando el formato explícito, porque una ruta corta confunde la detección por extensión.	etapa 8
osmium.geom.WKBFactory	Convierte las áreas comunales en geometrías WKB.	etapa 8

12.5 shapely

Geometría: puntos, líneas, polígonos y consultas espaciales.

Tabla 6. Funciones de shapely usadas en el proyecto

Función	Uso en el proyecto	Dónde
shapely.wkb.loads, shapely.to_wkb, shapely.from_wkb	Convierte geometrías desde y hacia WKB para guardarlas en el índice.	etapa 8
shapely.strtree.STRtree	Árbol espacial de polígonos comunales; responde qué comuna contiene cada punto.	etapa 8
STRtree.query, STRtree.query_nearest	Consultas de contención, intersección y vecino más cercano.	etapa 8
shapely.points, shapely.Point	Crea puntos de forma vectorizada o individual.	etapa 8
shapely.geometry.MultiLineString	Une los tramos de una calle o ruta.	etapa 8
MultiLineString.interpolate, .project	Punto medio de una calle y proyección de un punto sobre una ruta.	etapa 8
shapely.ops.nearest_points	Puntos más cercanos entre dos calles sin nodo común.	etapa 8

12.6 rapidfuzz

Similitud aproximada de cadenas.

Tabla 7. Funciones de rapidfuzz usadas en el proyecto

Función	Uso en el proyecto	Dónde
rapidfuzz.process.extract	Busca los mejores candidatos de nombre en la comuna.	etapa 8
rapidfuzz.fuzz.token_sort_ratio	Medida de similitud de la coincidencia aproximada de nombres.	etapa 8
rapidfuzz.fuzz.token_set_ratio	Junto con la anterior, promedio usado para las sugerencias de alias_sugeridos.csv.	etapa 8

12.7 pyarrow

Motor de lectura y escritura de Parquet que pandas usa internamente; el script verifica su presencia antes de la etapa 9.

12.8 openpyxl, pyxlsb, xlrd

Motores de lectura de Excel que pd.read_excel usa según la extensión (ENGINE_MAP).

12.9 streamlit

Interfaz web del dashboard.

Tabla 8. Funciones de streamlit usadas en el proyecto

Función	Uso en el proyecto	Dónde
st.set_page_config	Título, ícono y diseño ancho de la página.	dashboard
st.cache_resource	Guarda la tabla cargada una vez por proceso y la comparte entre sesiones.	dashboard
st.context.theme	Detecta si el visitante usa tema claro u oscuro.	dashboard
st.sidebar, st.header, st.divider	Estructura de la barra lateral de filtros.	dashboard
st.slider, st.selectbox, st.multiselect, st.segmented_control, st.radio, st.select_slider, st.number_input	Controles de filtro y configuración.	dashboard
st.title, st.caption, st.markdown, st.info, st.warning, st.error	Textos, notas, avisos y la hoja de estilos inyectada.	dashboard
st.columns, st.tabs	Distribución de indicadores y pestañas.	dashboard
st.metric	Tarjetas de indicadores.	dashboard
st.altair_chart, st.pydeck_chart	Muestran los gráficos y los mapas.	dashboard
st.dataframe, st.column_config.*	Tablas de puntos críticos y de calidad con formato por columna.	dashboard
st.download_button	Descarga de los puntos críticos en CSV.	dashboard
st.stop	Detiene la ejecución si faltan datos o ningún siniestro cumple los filtros.	dashboard

12.10 altair

Gráficos declarativos sobre Vega-Lite.

Tabla 9. Funciones de altair usadas en el proyecto

Función	Uso en el proyecto	Dónde
alt.Chart con mark_bar, mark_line, mark_rect, mark_text, mark_circle, mark_rule	Barras, líneas, matriz de calor, etiquetas y el indicador al pasar el cursor.	dashboard
alt.X, alt.Y, alt.Axis, alt.Scale, alt.Step	Ejes, escalas y alto por barra.	dashboard
alt.Tooltip, alt.Legend, alt.Color, alt.Opacity	Fichas emergentes, leyendas, color y atenuación del año parcial.	dashboard
alt.selection_point, alt.condition, alt.value	Indicador al pasar el cursor sobre la línea mensual.	dashboard

12.11 pydeck

Mapas con deck.gl.

Tabla 10. Funciones de pydeck usadas en el proyecto

Función	Uso en el proyecto	Dónde
pdk.Layer("ScatterplotLayer")	Puntos por siniestro y círculos de puntos críticos.	dashboard
pdk.Layer("HeatmapLayer")	Mapa de calor cuando hay más de 30.000 siniestros.	dashboard
pdk.ViewState, pdk.Deck	Vista inicial y composición del mapa.	dashboard

12.12 Biblioteca estándar de Python

Módulos incluidos en Python, sin instalación.

Tabla 11. Funciones de Biblioteca estándar de Python usadas en el proyecto

Función	Uso en el proyecto	Dónde
re.compile, re.sub, re.match, re.fullmatch, re.search, re.split, re.findall	Expresiones regulares de todas las reglas de limpieza y clasificación.	todas las etapas
unicodedata.normalize, unicodedata.combining	Quitan tildes por descomposición NFKD.	etapas 3, 4 y 8
difflib.get_close_matches	Aproximación de nombres de comuna y reparación del campo tribunal.	etapas 4 y 6
functools.lru_cache	Memoriza limpiar_vía y limpiar_ruta.	etapa 7
concurrent.futures.ThreadPoolExecutor, as_completed	Descargas en paralelo.	etapa 1
hashlib.sha256	Huellas de las planillas.	etapa 1
csv.DictWriter	Escribe el manifiesto de descargas.	etapa 1
os.replace, os.link, shutil.copy2, tempfile.gettempdir	Escrituras atómicas y ruta ASCII para osmium.	etapas 2 a 9
ctypes.windll.kernel32.GetShortPathNameW	Ruta corta 8.3 de Windows.	etapa 8
pickle.dump, pickle.load	Guardan y leen los índices OSM y MOP y la red MOP descargada.	etapa 8
bisect.bisect_left	Búsqueda binaria de la medida M en tramos crecientes.	etapa 8
math.cos, math.radians, math.hypot	Distancia equirectangular en metros.	etapa 8
collections.defaultdict, collections.Counter	Índices por comuna y contadores de diagnóstico.	etapas 7 y 8
logging.basicConfig, getLogger, FileHandler, StreamHandler, Formatter	Bitácora general y bitácoras propias de las etapas 7 y 8.	todas las etapas
random.randint, random.Random	Semilla de la corrida y muestras reproducibles.	inicio y etapa 8
importlib.util.find_spec	Verifica las bibliotecas opcionales antes de las etapas 8 y 9.	etapas 8 y 9
datetime.date, datetime.timedelta, datetime.now	Fechas de Excel, año máximo y fecha de generación.	etapas 1, 4 y 9
json.dumps, json.loads	Escriben y leen metadatos.json.	etapa 9, dashboard
gc.collect	Devuelve la memoria al final de cada interacción.	dashboard
time.time, time.sleep, sys.exit	Duraciones, esperas entre reintentos y código de salida.	todas las etapas

13. Constantes de configuración

Las funciones dependen de constantes definidas al inicio de cada sección del script, separadas de la lógica para que un ajuste no obligue a recorrer el código. Las principales son las siguientes.

Tabla 12. Constantes principales

Constante	Valor	Efecto
CSV_SEP, ENCODING	„ utf-8-sig	Formato de todos los CSV del pipeline.
YEAR_MIN	2010	Primer año descargado.
TIMEOUT_SEC, REINTENTOS, MAX_WORKERS	90, 3, 6	Tiempo máximo por solicitud, reintentos y descargas simultáneas.
FECHA_EXCEL	36.526 a 47.483	Números que se tratan como fechas de Excel y no como alturas.
OSM_INDICE_VERSION	7	Versión del índice OSM; al cambiarla el índice se reconstruye.
OSM_MAX_DIAS, MOP_MAX_DIAS	30, 180	Antigüedad máxima del extracto OSM y de la red MOP antes de descargarlos de nuevo.
UMBRAL_NOMBRE	88	Similitud mínima de la coincidencia aproximada (95 para claves de siete caracteres o menos).
CRUCE_MAX_M	40	Distancia máxima entre dos calles para aceptar un cruce sin nodo común.
NUM_MAX_DIF	200	Diferencia máxima de numeración para interpolar en la misma vereda.
PRECISION_M	10 a 5.000	Precisión estimada de cada método de ubicación, en metros.
DASH_PARTE_MB	23	Tamaño máximo de cada parte de datos del dashboard.
MAX_PUNTOS	30.000	Siniestros sobre los cuales el mapa pasa de puntos a densidad.
MIN_PUNTO_CRITICO	3	Siniestros mínimos para que una dirección sea punto crítico.

Fuente: elaboración propia, a partir de script.py y dashboard/app.py.